

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: XX hrs/min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Calidad y Excelencia	El estudiante aplica este valor al comprender que el software no solo debe funcionar correctamente (pruebas funcionales), sino que también debe cumplir con atributos de calidad como rendimiento, seguridad y usabilidad (pruebas no funcionales), buscando siempre la mejora continua del producto.
Compromiso	Se evidencia al reconocer la importancia de las pruebas End to End como mecanismo para garantizar que todas las integraciones del sistema funcionan en conjunto, asumiendo la responsabilidad de validar la experiencia completa del usuario, no solo componentes aislados.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Diseña y ejecuta estrategias de aseguramiento de la calidad del software, abarcando pruebas funcionales y no funcionales, con el fin de garantizar que el sistema cumpla con los estándares de rendimiento, seguridad, confiabilidad y satisfacción del usuario.
Competencia Específica	El estudiante desarrolla la capacidad de diferenciar e implementar distintos tipos de pruebas (unitarias, integración, E2E), identificar atributos de calidad asociados a pruebas no funcionales, y automatizar pruebas End to End utilizando la herramienta Cypress, interpretando correctamente los resultados de ejecución.

2. Palabras Clave

Pruebas No Funcionales: Validación de atributos de calidad del software que no están relacionados con comportamientos funcionales específicos, incluyendo rendimiento (performance), seguridad, usabilidad, confiabilidad, escalabilidad y compatibilidad.

Tipos de Pruebas (Niveles): Clasificación según el alcance y la granularidad: pruebas unitarias (componente individual), pruebas de integración (interacción entre componentes), pruebas de sistema (comportamiento completo) y pruebas de aceptación (validación con el usuario).

Prueba End to End (E2E): Tipo de prueba que verifica el flujo completo de una aplicación desde el inicio hasta el final, simulando el comportamiento de un usuario real e incluyendo todas las integraciones (bases de datos, APIs, servicios externos, interfaces de usuario).

Cypress: Herramienta de automatización de pruebas End to End para aplicaciones web modernas, que se ejecuta directamente en el navegador (sin Selenium), ofreciendo viajes en el tiempo (time travel), recargas automáticas y capturas de pantalla ante fallos.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada los conceptos fundamentales del aseguramiento integral de software, con énfasis en pruebas no funcionales, clasificación de tipos de pruebas y, principalmente, la automatización de pruebas End to End con la herramienta Cypress.

El asistente guiará a los estudiantes a través de:

1. Explicación conceptual de los diferentes tipos de pruebas (unitaria, integración, sistema, E2E) y su ubicación en la pirámide de pruebas.
2. Identificación y ejemplificación de pruebas no funcionales (rendimiento, seguridad, usabilidad, etc.) sobre un caso práctico.
3. Configuración del entorno Cypress desde cero (Node.js, instalación, estructura).
4. Escritura de una prueba E2E completa sobre una aplicación web demo (login + flujo de compra o tarea representativa).
5. Ejecución y análisis de resultados, incluyendo uso de comandos, aserciones y capturas de pantalla automáticas.

La clase debe orientarse a que el estudiante comprenda la diferencia entre niveles de prueba, la importancia de los atributos no funcionales, y adquiera habilidades prácticas para automatizar escenarios E2E con Cypress.

4.Contenido

Descripción del ejercicio

Se plantea un caso práctico: una **aplicación web de lista de tareas (To-Do App)** con funcionalidades de registro de usuario, inicio de sesión, creación de tareas, marcado de tareas como completadas y eliminación. El ejercicio consiste en:

1. Identificar qué tipo de pruebas aplicarían a diferentes componentes (unitarias, integración, E2E).
2. Reconocer atributos no funcionales críticos para la aplicación (tiempo de respuesta, concurrencia, usabilidad).
3. Automatizar una prueba End to End completa usando **Cypress** que cubra el siguiente flujo:
 - Registro de nuevo usuario.
 - Inicio de sesión.
 - Creación de 2 tareas.
 - Marcar una tarea como completada.
 - Validar que el contador de tareas se actualiza correctamente.
 - Cerrar sesión.

Objetivo de la demostración

Demostrar el procedimiento para planificar y automatizar pruebas End to End utilizando Cypress, integrando conceptos de tipos de pruebas y pruebas no funcionales, con el fin de que el estudiante comprenda cómo funciona un flujo completo de automatización, desde la instalación hasta la interpretación de resultados, y cómo se relaciona con la calidad integral del software.

Recursos necesarios

- **Software:**
 - Node.js (versión 16 o superior) y npm.
 - Cypress (versión 13 o superior) – `npm install cypress --save-dev`.
 - Navegador web (Chrome, Edge o Firefox).
 - Editor de código (Visual Studio Code recomendado).
- **Plataforma / Entorno:**
 - Aplicación demo: <https://example.cypress.io> (app de prueba de Cypress) o una aplicación To-Do local simple (el auxiliar puede usar `todomvc`).
- **Herramientas técnicas:**
 - Terminal / Línea de comandos (cmd, PowerShell, bash).

- Cypress Test Runner (interfaz gráfica).
- **Archivos de apoyo:**
 - Código base de la aplicación To-Do (si se usa local) o URL de la app pública.
 - Plantilla de especificación de prueba ([spec.js](#)).
- **Equipo necesario:** Computadora con acceso a internet (para instalar Cypress y acceder a la app demo).
- **Configuración mínima:** Node.js correctamente instalado (`node --version`, `npm --version`).

Desarrollo paso a paso

Paso 1

Acción inicial: Introducción a los tipos de pruebas y pruebas no funcionales (parte teórica aplicada).

Objetivo: Clarificar la taxonomía de pruebas y su aplicación práctica.

Qué observar: El auxiliar proyecta un cuadro resumen y lo relaciona con el caso de la To-Do App:

Nivel de prueba	Objeto de prueba	Ejemplo en To-Do App	¿Automatizable?
Unitaria	Función/método individual	Validar que <code>calcularTareasPendientes()</code> devuelve el valor correcto	Sí (Jest, Mocha)
Integración	Interacción entre dos o más módulos	Verificar que el servicio de API registra usuario y retorna token	Sí (Supertest, Cypress)
Sistema	Aplicación completa	Probar que la interfaz web funciona con la base de datos real	Sí (Cypress, Selenium)
End to End (E2E)	Flujo completo + sistemas externos	Registro → login → crear tarea → completar → cierre sesión	Sí (Cypress)

Pruebas no funcionales (ejemplos para la To-Do App):

Atributo	Pregunta clave	Herramienta sugerida
Rendimiento (performance)	¿La app responde en menos de 2 segundos con 100 usuarios concurrentes?	Lighthouse, k6, JMeter
Seguridad	¿La contraseña se transmite cifrada? ¿Hay protección contra XSS/CSRF?	OWASP ZAP, Burp Suite
Usabilidad	¿El usuario puede crear una tarea en menos de 10 segundos sin instrucciones?	Pruebas manuales, feedback de usuarios
Escalabilidad	¿La app sigue funcionando si hay 10,000 tareas almacenadas?	Pruebas de carga con k6

Validación importante: Los estudiantes deben entender que las pruebas E2E son las más costosas (tiempo, recursos) y por eso se ubican en la cima de la pirámide de pruebas (pocas pero de alto valor).

Paso 2

Acción: Configuración del entorno Cypress.

Explicación de comandos (en vivo en terminal):

```
bash
```

```
# Verificar Node.js instalado
```

```
node --version # Debe mostrar v16+
```

```
# Crear carpeta del proyecto
```

```
mkdir demo-cypress-e2e
```

```
cd demo-cypress-e2e
```

```
# Inicializar proyecto npm
```

```
npm init -y
```

Instalar Cypress como dependencia de desarrollo

```
npm install cypress --save-dev
```

Abrir Cypress por primera vez (creará estructura de carpetas)

```
npx cypress open
```

Lo que se espera observar: Cypress crea automáticamente las carpetas `e2e`, `fixtures` y `support`. Se abre el Cypress Test Runner (ventana gráfica). El auxiliar explica la función de cada carpeta.

Error común destacado: Algunas versiones de Node.js antiguas (v14 o menor) generan conflictos. Si ocurre error, actualizar Node.js.

Paso 3

Ejecución principal (parte A): Escritura de una prueba E2E completa en Cypress.

Detalle: El auxiliar crea un archivo `to-do-flow.cy.js` dentro de la carpeta `cypress/e2e/` y escribe el siguiente código (explicando línea por línea):

```
javascript
```

```
/// <reference types="cypress" />
```

```
describe('Flujo End to End - Aplicación To-Do (Cypress demo)', () => {
```

```
  beforeEach(() => {
```

```
    // Visitar la aplicación demo ANTES de cada prueba
```

```
    cy.visit('https://example.cypress.io/todo')
```

```
  })
```

```
  it('Debe permitir registro, login y gestión de tareas', () => {
```

```
    // ===== PASO 1: Registro de nuevo usuario (simulado) =====
```

```
    // En esta app demo no hay registro real; simulamos un alias de usuario
```

```
    // para efectos demostrativos. En una app real usaríamos cy.request() para API.
```

```
    const testUser = `usuario_${Date.now()}@cypress.com`
```

```
// ===== PASO 2: Crear dos tareas nuevas =====  
  
// Primera tarea  
cy.get('[data-testid="new-todo"]').type('Comprar leche{enter}')  
cy.get('[data-testid="todo-item"]').should('contain', 'Comprar leche')  
  
// Segunda tarea  
cy.get('[data-testid="new-todo"]').type('Estudiar para examen de software{enter}')  
cy.get('[data-testid="todo-item"]').should('have.length', 2)  
  
// ===== PASO 3: Marcar la primera tarea como completada =====  
cy.get('[data-testid="todo-item"]').first()  
  .find('.toggle')  
  .click()  
  
// ===== PASO 4: Verificar que el contador de tareas pendientes es 1 =====  
cy.get('[data-testid="todo-count"]').should('contain', '1 item left')  
  
// ===== PASO 5: Filtrar tareas completadas y verificar =====  
cy.contains('Completed').click()  
cy.get('[data-testid="todo-item"]').should('have.length', 1)  
cy.get('[data-testid="todo-item"]').should('contain', 'Comprar leche')  
  
// ===== PASO 6: Filtrar activas (pendientes) y verificar =====  
cy.contains('Active').click()  
cy.get('[data-testid="todo-item"]').should('have.length', 1)  
cy.get('[data-testid="todo-item"]').should('contain', 'Estudiar para examen de software')  
  
// ===== PASO 7: Limpiar tareas completadas =====  
cy.contains('Clear completed').click()  
cy.get('[data-testid="todo-item"]').should('have.length', 1)
```



```

    cy.get('[data-testid="todo-item"]').should('contain', 'Estudiar para examen de software')
  })
})

```

Interpretación de resultado parcial: Mientras se ejecuta la prueba, el Test Runner de Cypress muestra en tiempo real cada acción (navegación, escritura, clics, aserciones). Si todo es correcto, todas las aserciones pasan y se muestra un resultado **verde**.

El auxiliar explica los comandos clave:

- `cy.visit(url)`: navega a una URL.
- `cy.get(selector)`: selecciona elemento(s) del DOM.
- `.type(texto)`: simula escritura en un input.
- `.click()`: simula clic en botón/checkbox.
- `.should('contain', texto)`: aserción de que el elemento contiene texto.
- `.should('have.length', n)`: aserción de cantidad de elementos.

Atención: El selector `[data-testid="..."]` es una buena práctica para pruebas E2E (evita romper pruebas cuando cambian clases CSS o IDs).

Paso 3 (continuación)

Ejecución principal (parte B): Manejo de pruebas no funcionales dentro de Cypress (opcional/avanzado).

Detalle: El auxiliar muestra cómo Cypress permite validar ciertos atributos no funcionales básicos:

javascript

```

// Prueba de rendimiento (tiempo de carga)
it('La página debe cargar en menos de 3 segundos', () => {
  cy.visit('https://example.cypress.io/todo', {
    timeout: 3000 // falla si excede 3 segundos
  })
})

// Prueba de usabilidad (simular pantalla móvil)
it('Los botones deben ser visibles en resolución móvil', () => {

```

```
cy.viewport('iphone-x')
cy.visit('https://example.cypress.io/todo')
cy.get('[data-testid="new-todo"]').should('be.visible')
})

// Prueba de accesibilidad (básica) - requiere plugin adicional
// El auxiliar menciona cypress-axe para pruebas de accesibilidad.
```

Explicación: Aunque Cypress es principalmente para pruebas funcionales E2E, se puede integrar con otras herramientas para pruebas no funcionales avanzadas (accesibilidad, rendimiento con Lighthouse via CLI, etc.).

Paso 4

Cierre técnico: Ejecución completa, análisis de resultados y buenas prácticas.

Acciones finales:

1. Ejecutar la prueba en modo **headless** (sin interfaz gráfica) para simular CI/CD:
bash
npx cypress run
2. Mostrar el video generado automáticamente (carpeta **cypress/videos**) y las capturas de pantalla ante fallos.
3. Explicar el **time-travel** de Cypress: al hacer clic en cada paso del Test Runner, se puede ver exactamente el estado de la aplicación en ese momento.
4. Resumir las buenas prácticas para E2E con Cypress:
 - Usar **data-testid** en lugar de clases o IDs volátiles.
 - Mantener las pruebas independientes (una no depende del estado de otra).
 - Usar **cy.intercept()** para simular respuestas de API (evitar dependencias externas).
 - Evitar **cy.wait(segundos)**; preferir **cy.wait('@alias')** para esperar peticiones.

Confirmación de aprendizaje: Los estudiantes, de forma individual, escriben una prueba E2E simple que:

- Visite la misma aplicación demo.
- Cree exactamente **una tarea**.
- Marque esa tarea como completada.
- Verifique que el contador muestre "0 items left".

El auxiliar revisa en vivo o da feedback al final del ejercicio.

Observaciones importantes

- **Error común:** Usar `cy.wait(5000)` en lugar de esperar condiciones reales (como `cy.get('.element', { timeout: 10000 })`). Esto hace que las pruebas sean lentas y frágiles.
- **Error común:** Escribir pruebas E2E que dependen de datos existentes (ej. asumir que hay exactamente 3 tareas). Siempre limpiar el estado antes de cada prueba con `beforeEach`.
- **Buena práctica:** Organizar los comandos de Cypress usando **Custom Commands** (ej. `cy.login('user', 'pass')`) para evitar repetir código.
- **Advertencia sobre pruebas no funcionales:** Cypress NO es la herramienta adecuada para pruebas de carga/estrés (eso es k6, JMeter) ni para pruebas de seguridad profundas (usar OWASP ZAP). Explicar las limitaciones.
- **Recomendación:** Instalar `cypress-localstorage-commands` si se requiere mantener sesión entre pruebas.

Tabla 1 – Comparativa de Tipos de Pruebas y Herramientas Asociadas

Elemento	Pruebas Unitarias	Pruebas de Integración	Pruebas E2E
Alcance	Función / método individual	Interacción entre 2+ módulos	Flujos completos de usuario
Velocidad	Muy rápida (milisegundos)	Rápida (segundos)	Lenta (minutos)
Mantenimiento	Bajo	Medio	Alto (frágiles a cambios de UI)
Costo de fallo	Bajo	Medio	Alto (puede detener despliegue)

Herramientas JS	Jest, Mocha, Vitest	Supertest, Cypress (component testing)	Cypress, Playwright, Selenium
¿Prueba integraciones?	No (se mockean dependencias)	Sí (parcial)	Sí (completo con sistemas reales)
Ubicación pirámide	Base (muchas)	Medio (algunas)	Cima (pocas)

Resultado esperado

Al finalizar el ejercicio, cada estudiante debe ser capaz de:

1. **Clasificar** correctamente un requerimiento de prueba como unitario, integración, sistema o E2E.
2. **Identificar** al menos 3 atributos de calidad asociados a pruebas no funcionales y mencionar una herramienta para evaluar cada uno.
3. **Instalar y configurar** Cypress desde cero en un proyecto Node.js.
4. **Escribir y ejecutar** una prueba E2E completa que interactúe con una aplicación web real, usando comandos de Cypress (`cy.visit`, `cy.get`, `.type`, `.click`, `.should`).
5. **Interpretar** los resultados del Test Runner (verde = éxito, rojo = fallo) y usar las capturas de pantalla para depurar errores.
6. **Explicar** por qué las pruebas E2E son la cima de la pirámide y en qué casos priorizar pruebas más rápidas (unitarias/integración).

Aplicación práctica

Este ejercicio se relaciona directamente con:

- **Ciclo de vida de desarrollo profesional:** Las pruebas E2E automatizadas son ejecutadas en pipelines CI/CD (GitHub Actions, Jenkins, GitLab CI) antes de cada despliegue.
- **Integración con otras prácticas:** Cypress se puede integrar con Cucumber (BDD), con reportes como Mochawesome, y con dashboards de Cypress Cloud.
- **Cursos posteriores:** DevOps, Integración Continua, Calidad de Software, Pruebas de Aceptación Automatizadas.

- **Casos reales:** Empresas como Shopify, NASA, DHL, y muchas startups usan Cypress para sus suites de pruebas E2E.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- La instalación de Node.js y Cypress consume tiempo si los estudiantes no tienen permisos de administrador o tienen versiones antiguas. Varios fallaron en el `npm install` por problemas de red o caché.
- El concepto de "asincronía" en Cypress es confuso: `cy.get` no retorna un elemento directamente (ni una promesa estándar), sino un objeto "Chainable". Esto causa errores si los estudiantes intentan guardar elementos en variables (ej. `const boton = cy.get('button')`).
- Los selectores `data-testid` no estaban presentes en todas las aplicaciones demo, lo que obligó a usar selectores frágiles (clases dinámicas).
- Los estudiantes no comprenden inmediatamente la diferencia entre una prueba E2E fallida por un bug real vs fallida por un selector incorrecto.

Recomendaciones para futuras sesiones:

- **Preparar un entorno "contenedorizado"** (Docker con Node.js + Cypress ya instalado) para evitar problemas de instalación en equipos diversos.
- **Usar una aplicación demo propia** (un repositorio con un pequeño To-Do App que tenga `data-testid` bien definidos) en lugar de depender de páginas externas que pueden cambiar.
- **Grabar un video corto (10 min)** de la instalación paso a paso y compartirlo 48 horas antes, para que los estudiantes lleguen con el entorno listo.
- **Proveer un "cheatsheet" de comandos Cypress** impreso o en PDF para consulta rápida.
- **Enfatizar que las pruebas E2E deben ser las menos numerosas** (regla 80/20: 80% unitarias, 15% integración, 5% E2E).

Qué puntos consideras fue de mayor dificultad para el estudiante:

- **La asincronía implícita de Cypress:** el mecanismo de reintentos (retry-ability) cuesta de comprender. Los estudiantes esperan un flujo lineal al estilo "instrucción 1 → instrucción 2".
- **Escribir aserciones correctas:** a menudo usaban `.should('have.value')` cuando el elemento era un div en lugar de un input.
- **Depurar pruebas fallidas:** no sabían leer las capturas de pantalla automáticas ni el "Command Log" de Cypress para identificar el paso exacto del fallo.
- **Diferenciar pruebas no funcionales de funcionales:** muchos asumían que "rendimiento" se podía probar solo con `cy.visit()` con timeout, sin entender que las pruebas reales de rendimiento requieren métricas más sofisticadas (percentiles 95/99, throughput, etc.).
- **Manejo de estado entre pruebas:** no limpiaban el localStorage o las cookies, lo que hacía que una prueba contaminara a otra.

6. Referencias

1. [Cypress.io](https://docs.cypress.io/guides/end-to-end-testing/writing-your-first-end-to-end-test). (2024). *Cypress Documentation | End to End Testing*. Recuperado de <https://docs.cypress.io/guides/end-to-end-testing/writing-your-first-end-to-end-test>
2. Myers, G., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3ª ed.). Wiley. (Capítulos sobre pruebas funcionales vs no funcionales).
3. Fowler, M. (2021). *The Practical Test Pyramid*. Recuperado de <https://martinfowler.com/articles/practical-test-pyramid.html>
4. npm, Inc. (2024). *Cypress package*. Recuperado de <https://www.npmjs.com/package/cypress>
5. OWASP Foundation. (2024). **OWASP Testing Guide (v4) - Non-Functional Testing**. Recuperado de <https://owasp.org/www-project-web-security-testing-guide/>
6. Google Chrome Labs. (2024). *Lighthouse CI | Performance testing*. Recuperado de <https://github.com/GoogleChrome/lighthouse-ci>